

An Evidence-First Plan to Fine-Tune Your First Open LLM

Prepared for: Bhargav Mahadevan

Purpose: Learn how LLM fine-tuning works, create a real training record, and generate honest early demonstration data for GhostLayer without asking a client to trust you first.

Author: Manus AI

The right first goal

You should **not** try to train a foundation model from scratch as your first project. That would mean learning tokenization, distributed training, data curation, massive compute management, evaluation, and deployment all at once. Instead, start by **fine-tuning an existing small open instruct model** on a narrow task using LoRA or QLoRA.

Your first win is not “I built a new ChatGPT.” Your first win is: **“I ran a reproducible, documented fine-tuning experiment; I can show what I changed, what happened, and what I still do not know.”**

Parameter-efficient fine-tuning methods train a small number of additional parameters instead of every model weight, which makes adaptation more accessible on consumer-oriented hardware. ¹ TRL’s supervised fine-tuning workflow accepts conversational or prompt/completion datasets, so it is a good learning path for turning a small collection of examples into a working specialized assistant. ²

Decision	Recommended first choice	Why it is the sensible starting point
Training type	Supervised fine-tuning (SFT)	It is the most direct way to learn the full data → train → evaluate loop.
Adaptation method	LoRA or QLoRA	It keeps the experiment smaller and preserves the base model.
Model size	Roughly 0.5B–3B instruct model	Small enough for a first experiment; choose a model with a license you understand.
First task	Turn real run records into a short structured training-review note	It connects directly to GhostLayer without pretending to optimize anything automatically.

First claim	“We can produce an auditable review record from supplied run context.”	This is demonstrable and does not overclaim efficiency gains.
-------------	--	---

Your first project: the Training Review Assistant

Build a small assistant whose job is deliberately narrow: given a structured record from **one of your own training runs**, it produces a careful review note. For example, it can summarize the run configuration, identify what changed versus a comparison run, identify missing evidence, and suggest one human-reviewed next test. It must never claim that it changed a GPU setting or proved a savings outcome.

The important distinction is that you are **not** asking the fine-tuned model to be a magical optimizer. You are teaching it a consistent output format for reviewing evidence. This makes the work useful for GhostLayer while keeping its claim boundaries honest.

The data you should collect yourself

Run a small training workload that you own and can repeat. A small language-model fine-tuning run is ideal, but the initial workload can also be a compact text-classification or next-token experiment. Record the same fields for every run.

Field	Example	Why it matters
run_id	gl-demo-001	Connects every output to a real experiment.
model_id and revision	Selected open instruct model + exact revision	Makes the starting point reproducible.
dataset_id and license	Your dataset version and usage permission	Prevents unclear provenance.
seed	A fixed integer	Makes comparisons more meaningful.
hardware	GPU type, VRAM, runtime environment	Gives hardware context without inventing telemetry.
training_config	Learning rate, batch size, epochs, sequence length, LoRA settings	Records what actually changed.
metrics	Training loss, validation loss, wall-clock time, tokens,	Supplies measurable outcomes.

	evaluation score	
<code>raw_log_path</code> and hash	Link or digest of the raw console log	Lets you verify the summary later.
<code>review_note</code>	A human-written explanation of what to inspect next	Becomes the supervised target for the first fine-tune.

Do **not** populate these records with invented metrics. If a value was not measured, write `unknown` . If it is estimated, mark it as `estimated` . If it comes from a browser demonstration, mark it as `simulated` .

A 30-day execution plan

Days 1–3: learn the smallest end-to-end loop

Choose one small open instruct model after reading its current model card and license. Use a beginner notebook or local setup to complete a smoke-test fine-tune on an openly licensed example dataset. This is a learning run only; do not use it in GhostLayer marketing. Hugging Face’ s SFT workflow includes a minimal example using a small Qwen model and supports conversational datasets directly. ² Unsloth documents a beginner path using small instruct models, LoRA/QLoRA, notebooks, and local environments. ³

Your deliverable is a folder containing: the notebook, environment versions, model identifier, dataset identifier, one run log, and a short note describing what you learned.

Days 4–10: create your own real experiment records

Run a compact workload at least 20 times with controlled changes. Change **one variable at a time**—for example, sequence length, learning rate, batch size, or LoRA rank. Keep a baseline configuration unchanged and give every run a unique ID.

Write a brief human review note after each run. The note should follow a stable pattern:

1. **Observation:** What does the record show?
2. **Boundary:** What is missing or uncertain?
3. **Next test:** What one controlled comparison is reasonable?
4. **Claim status:** Measured, estimated, simulated, or unknown.

This work is more important than adding hundreds of weak examples. A small, truthful dataset with carefully written labels teaches you far more about the actual problem.

Days 11–14: build the dataset and holdout set

Convert the records into a conversational JSONL dataset. Each input should contain only the structured run context that you are willing to show. Each expected assistant response should be the review note you wrote from that record. Keep at least 20% of your records aside as a **holdout evaluation set**; do not train on them.

Use a schema such as the following:

JSON

```
{
  "messages": [
    {"role": "user", "content": "Review this training record: {\\"run_id\\": \\"gl
    {"role": "assistant", "content": "Observation: validation loss is lower tha
  ]
}
```

TRL supports conversational records in this `messages` shape and can apply a model’s chat template during supervised fine-tuning. 2

Days 15–18: run the first controlled fine-tune

Start with LoRA or QLoRA rather than full fine-tuning. PEFT is built for adapting pretrained models by training a small number of extra parameters, and it integrates with the core Hugging Face training stack. 1 Keep your first training configuration intentionally boring: a small model, a modest context length, fixed seed, a single controlled dataset split, and one saved configuration file.

Save three things after the run:

Artifact	What it proves
Base-model responses to the holdout prompts	Your starting behavior.
Fine-tuned-model responses to the same holdout prompts	Whether the fine-tune changed the targeted behavior.
Run manifest and logs	What you actually ran.

Days 19–22: evaluate without fooling yourself

Compare the base model and the fine-tuned model on the held-out records. Use a manual rubric with four pass/fail checks:

Evaluation check	Pass condition
Factual fidelity	Every quoted metric appears in the supplied input record.
Boundary discipline	The response labels uncertainty and does not invent a savings or hardware claim.
Actionability	The next test changes one clear variable.
Format consistency	The note contains Observation, Boundary, Next test, and Claim status.

Do not rely only on training loss. The training workflow can log loss, token count, learning rate, gradient norm, and other training metrics, but a lower loss alone does not prove that a review note is useful or trustworthy. 2 The held-out rubric is where you decide whether the model has improved on the behavior you care about.

Days 23–30: turn the experiment into the first GhostLayer case study

Publish an **evidence packet**, not a performance claim. It can live on a private page at first and should include:

1. The plain-language goal: produce clear review notes from supplied run records.
2. The exact model, dataset, hardware, and configuration used.
3. A table of baseline versus fine-tuned holdout examples.
4. The evaluation rubric and results.
5. A limitations statement: no client data, no live telemetry ingestion, no autonomous apply/revert, and no verified GPU savings claim.
6. The next experiment you will run.

This is the bridge between your personal technical work and GhostLayer. It demonstrates that you care about the same things your website promises: provenance, reviewability, and claim boundaries.

The experiment tracker GhostLayer should eventually show

Your current website can become more credible when you can replace illustrative UI with a single real case-study record. Do not build a full dashboard yet. Start with one page that shows the fields below.

Panel	First real data to show
Run record	Run ID, model revision, dataset revision, seed, hardware, timestamp.
Change log	One variable changed and the reason it was changed.
Evidence	Raw-log hash, training/validation metrics, wall-clock time, result label.
Review note	The base response, fine-tuned response, and human evaluation.
Boundary	What the experiment does not establish.

What you should not do yet

Do not train a large foundation model from scratch. Do not scrape data whose license or consent is unclear. Do not fine-tune on client data without written permission and a defined security model. Do not market one run as a GPU-efficiency breakthrough. Do not build autonomous tuning, apply, or rollback interfaces before you have a trustworthy record and evaluation loop.

The disciplined move is smaller: run a real experiment, keep its receipts, explain the result in plain language, then repeat.

A simple starting stack

Use a hosted notebook or a local NVIDIA setup, an open instruct model you have reviewed for license compatibility, a small JSONL dataset of your own run records, Hugging Face TRL plus PEFT, and a plain folder structure for your artifacts. Unsloth is an optional beginner-friendly route if you prefer notebook-guided LoRA/QLoRA workflows. ³

First milestone: By the end of the first week, you should be able to point to one committed notebook, one small dataset, one completed adapter run, and one honest evaluation table. That is enough to begin building real GhostLayer evidence.

References

- [1] Hugging Face PEFT documentation
- [2] Hugging Face TRL SFTTrainer documentation

[3] Unsloth Fine-Tuning LLMs Guide